۱- روشن و خاموش کردن LED با استفاده از وقفه

با استفاده از وقفه برنامه ای بنویسید که با فشردن کلید، LED روشن و با فشردن دوباره آن LED خاموش شود.

شرح گزارش کار :

I. در کدویژن یک پروژه جدید ایجاد میکنیم. در پنجره کد ویزارد چیپ ATMEGA16 را انتخاب میکنیم. پایه PORTC0 را برای وصل کردن LED ، خروجی تعریف می کنیم. در قسمت External Interrupts تیک INT0 را می زنیم و Mode آن را روی Rising Edge میگذاریم. با اینکار وقفه ۰ فعال می شود و در هر لبه بالارونده در هنگام فشردن کلید روتین آن اجرا می شود. بقیه تنظیمات را تغییر نمی دهیم. مطمئن میشویم تیک گزینه Program > Generate Code For Disabled Peripherals برداشته شده است. سپس گزینه Program > Generate, Save and Exit را میزنیم. در سه پنجره باز شده برای ذخیره کردن فایل های پروژه همه ی فایل ها را در یک فولدر و در کنار هم ذخیره میکنیم.

II. تابع ext_int0_isr به طور خودکار ایجاد می شود. این تابع در هر لبه بالارونده وقفه ۰ فراخوانی می شود. بنابراین باید کد مربوط به روشن و خاموش شدن LED با کلید فشاری را در این تابع بنویسیم. رجیستر های مربوط به وقفه ها به طور خودکار تنظیم شده اند بنابراین نیازی به کنترل آن ها نیست. داخل روتین وقفه ابتدا از یک delay_ms(100) برای خنثی کردن نوسانات کلید استفاده میکنیم. در هر بار فشرده شدن کلید باید وضعیت معکوس LED در PORTC.0 نوشته شود تا اگر LED روشن بود خاموش شود و اگر خاموش بود روشن شود. برای اینکار خط PORTC.0=!PORTC.0 را به روتین وقفه اضافه می کنیم. در پایان باید پرچم مربوط به وقفه را فعال کنیم. اینکار را با اضافه کردن دستور GICR |= (1 << INTF0) انجام میدهیم. از منوی Project > Build All برنامه را کامپایل میکنیم.

نحوه کار برنامه به این صورت است که اگر کلیدی فشرده شده باشد وضعیت روشن/خاموش بودن LED را معکوس می کند.

III. در پروتئوس یک پروژه جدید با تنظیمات پیش فرض ویزارد ایجاد میکنیم. کلید فشاری، LED و میکروکنترلر ATMEGA16 را با جستجو در Device ها پیدا میکنیم و به پروژه اضافه میکنیم. LED را با کمک مقاومت محدودکننده جریان به PC0 و از آن طرف به زمین متصل میکنیم. کلید فشاری را نیز از یک طرف به صورت Pull-Down به پایه PD2/INT0 و از آن طرف به POWER وصل میکنیم.روی میکرو دابل کلیک میکنیم. در پنجره باز شده بخش Program File کد هگز برنامه تولید شده در کدویژن را روی میکرو پروگرم میکنیم. با اجرای شبیه ساز مشاهده میکنیم که هرگاه کلید فشرده شود اگر LED خاموش باشد روشن می شود و اگر LED روشن باشد خاموش می شود.

۲- نمایش کلید فشرده شده کیپد روی سون سگمنت با استفاده از وقفه صفر

شرح گزارش کار :

I. در کدویژن یک پروژه جدید ایجاد میکنیم. در پنجره کد ویزارد چیپ ATMEGA32 را انتخاب میکنیم. در تب Ports پورت B را انتخاب میکنیم و همه ی بیت های آن را روی حالت Out یا همان ۱ میگذاریم تا بتوانیم روی آن بنویسیم و سون سگمنت را به آن وصل کنیم. ۳ بیت کم ارزش PORTC را ورودی با مقاومت بالاکش و ۴ بیت با ارزش آن را خروجی تعیین می کنیم تا کیپد را به آن وصل کنیم. در قسمت External Interrupts تیک INT0 را می زنیم و Mode آن را روی Any Change میگذاریم تا با هر تغییری روتین وقفه اجرا شود. بقیه تنظیمات را تغییر نمیدهیم. مطمئن میشویم تیک گزینه Program > Generate Code For Disabled Peripherals برداشته شده است. سپس گزینه Program > Generate, Save and Exit را میزنیم. در سه پنجره باز شده برای ذخیره کردن فایل های پروژه همه ی فایل ها را در یک فولدر و در کنار هم ذخیره میکنیم.

II. در بالای برنامه آرایه sevenSegment را تعریف میکنیم و کد هگز اعداد 0 تا 9 برای سون سگمنت کاتد مشترک را در آن ذخیره میکنیم. متغیر key را نیز برای ذخیره عدد فشرده شده تعریف میکنیم. تابع GetKey() را برای خواندن عدد فشرده شده در کیپد به این صورت تعریف می کنیم که به سطر های کیپد PORTC4…7 عدد ۱ منتسب می شود و سپس ۴ بار و در هر بار یک سطر را ۰ میکنیم و در این حین تک تک ستون های کیپد PINC0…2 را میخوانیم و اگر هر کدام ۰ شد یعنی کلید متناظر با آن ستون و سطری که ۰ کردیم فشرده شده پس عدد آن را بر میگردانیم. در پایان این تابع اگر هیچکدام از کلیدها فشرده نشده بود عدد 1- را بر میگردانیم. تابع ext_int0_isr به طور خودکار ایجاد می شود. این تابع با هر تغییر در پایه وقفه ۰ فراخوانی می شود. داخل روتین وقفه char k را برای ذخیره کردن کلید فشرده شده تعریف می کنیم. با دستور k=GetKey() کلید فشرده شده را میخوانیم و با دستور شرطی بررسی میکنیم. اگر k!=-1 بود یعنی کلیدی فشرده شده بود کد هگز مربوط به عدد فشرده شده را در PORTB می نویسد. برای اینکه نیازی نباشد تا کلید را نگه داریم تا سون سگمنت روشن بماند این چند خط را در شرط if(!PIND.2) میگذاریم. به این ترتیب فقط زمانی که پایه وقفه تغییر وضعیت دهد شماره کلید روی سون سگمنت نوشته می شود. در پایان باید پرچم مربوط به وقفه را فعال کنیم. اینکار را با اضافه کردن دستور GICR |= (1 << INTF0) انجام میدهیم. از منوی Project > Build All برنامه را کامپایل میکنیم.

نحوه کار برنامه به این صورت است که با هر بار فشرده شدن کلید لبه پایین رونده در INT0 ایجاد می شود. بنابراین روتین وقفه فراخوانی میشود. در هر بار اجرای روتین وقفه کیپد اسکن میشود و اگر کلیدی فشرده شده بود آن را در سون سگمنت نمایش میدهد.

III. در پروتئوس یک پروژه جدید با تنظیمات پیش فرض ویزارد ایجاد میکنیم.کامپوننت های سون سگمنت کاتد مشترک، کیپد ۴*۳ و میکروکنترلر ATMEGA32 را با جستجو در Device ها پیدا میکنیم و به پروژه اضافه میکنیم. پایه های دیتای سون سگمنت را به پورت B وصل میکنیم. کاتد مشترک را به زمین متصل میکنیم. ستون های ۱ تا ۳ کیپد را به بیت های 0…2 و سطرهای A تا D را به بیت های 4…7 از پورت C وصل میکنیم. همچنین هر یک از پایه های ستون ها را به یک دیود (روی حالت Digital) وصل میکنیم. سمت دیگر دیودها را به یکدیگر وصل میکنیم و سپس به شکل Pull-Up شده با یک مقاومت به پایه PD2/INT0 وصل میکنیم. به این صورت هرگاه کلیدی فشرده شود این پایه ۰ می شود و در حالت عادی ۱ است. روی میکرو دابل کلیک میکنیم. در پنجره باز شده بخش Program File کد هگز برنامه تولید شده در کدویژن را روی میکرو پروگرم میکنیم. با اجرای شبیه ساز مشاهده میکنیم که سون سگمنت مقدار هر کلید فشرده شده را نمایش میدهد.

پیوست ها:

آزمایش ۱)

G:\Personal\Uni\Micro Lab\Micro\Az6-1\Az6-1.c

Notes | Az6-1.c

```
/*****************************************************
This program was created by the
CodeWizardAVR V3.12 Advanced
Automatic Program Generator
© Copyright 1998-2014 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project : Session6-Experiment1
Version :
Date    : 11/9/2020
Author  : Ali Sepehrnia (96121038)
Company :
Comments:


Chip type               : ATmega16
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 256
*****************************************************/

#include <mega16.h>
#include <delay.h>

// Declare your global variables here

// External Interrupt 0 service routine
interrupt [EXT_INT0] void ext_int0_isr(void)
{
    delay_ms(100);
    PORTC.0=!PORTC.0;
}

```

G:\Personal\Uni\Micro Lab\Micro\Az6-1\Az6-1.c

Notes | Az6-1.c

```
void main(void)
{
// Declare your local variables here

// Input/Output Ports initialization
// Port A initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

// Port B initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRB=(0<<DDB7) | (0<<DDB6) | (0<<DDB5) | (0<<DDB4) | (0<<DDB3) | (0<<DDB2) | (0<<DDB1) | (0<<DDB0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

// Port C initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=Out
DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (1<<DDC0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=0
PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

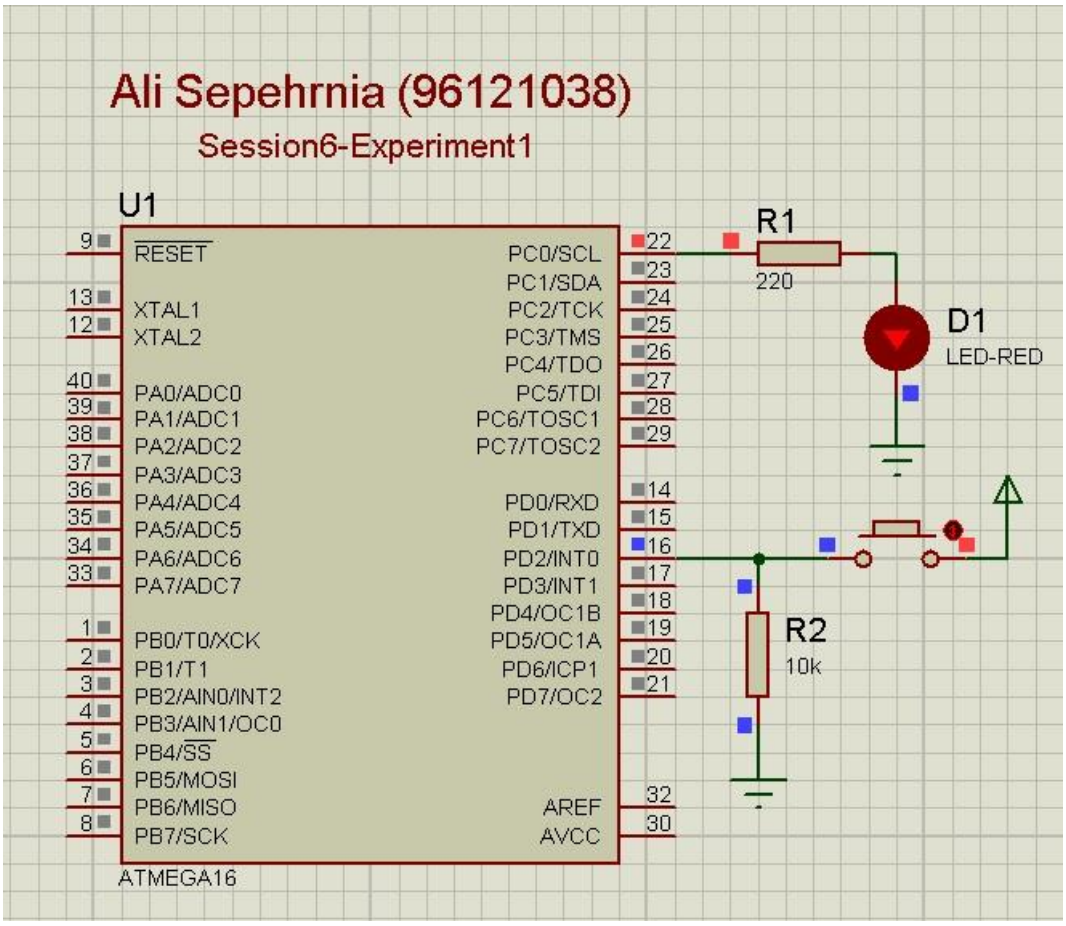
// Port D initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
// State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

// External Interrupt(s) initialization
// INT0: On
// INT0 Mode: Rising Edge
// INT1: Off
// INT2: Off
GICR|=(0<<INT1) | (1<<INT0) | (0<<INT2);
MCUCR=(0<<ISC11) | (0<<ISC10) | (1<<ISC01) | (1<<ISC00);
```

G:\Personal\Uni\Micro Lab\Micro\Az6-1\Az6-1.c

Notes | Az6-1.c

```
71  MCUCR=(0<<ISC11) | (0<<ISC10) | (1<<ISC01) | (1<<ISC00);
72  MCUCSR=(0<<ISC2);
73  GIFR=(0<<INTF1) | (1<<INTF0) | (0<<INTF2);
74
75  // Global enable interrupts
76  #asm("sei")
77
78  while (1)
79        {
80        // Place your code here
81
82        }
83  }
84
```



آزمایش ۲)

G:\Personal\Uni\Micro Lab\Micro\Az6-2\Az6-2.c

Notes | Az6-2.c

```
/*****************************************************
This program was created by the
CodeWizardAVR V3.12 Advanced
Automatic Program Generator
© Copyright 1998-2014 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project : Session6-Experiment2
Version :
Date    : 10/30/2020
Author  : Ali Sepehrnia (96121038)
Company :
Comments:


Chip type               : ATmega32
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 512
*****************************************************/

#include <mega16.h>
#include <delay.h>

// Declare your global variables here
int sevenSegment[] = {0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f};

int GetKey()
{
    char k=-1;

    PORTC=0xff;
```

G:\Personal\Uni\Micro Lab\Micro\Az6-2\Az6-2.c

Notes | Az6-2.c

```
    PORTC.4 = 0;
    if(PINC.0 == 0) k=1;
    else if(PINC.1 == 0) k=2;
    else if(PINC.2 == 0) k=3;
    PORTC.4 = 1;

    PORTC.5 = 0;
    if(PINC.0 == 0) k=4;
    else if(PINC.1 == 0) k=5;
    else if(PINC.2 == 0) k=6;
    PORTC.5 = 1;

    PORTC.6 = 0;
    if(PINC.0 == 0) k=7;
    else if(PINC.1 == 0) k=8;
    else if(PINC.2 == 0) k=9;
    PORTC.6 = 1;

    PORTC.7 = 0;
    if(PINC.1 == 0) k=0;
    PORTC.7 = 1;

    PORTC=0x0f;

    return k;
}

// External Interrupt 0 service routine
interrupt [EXT_INT0] void ext_int0_isr(void)
{
char k;
//delay_ms(200);
if(!PIND.2)
    {
        k = GetKey();
        if(k != -1)
```

G:\Personal\Uni\Micro Lab\Micro\Az6-2\Az6-2.c

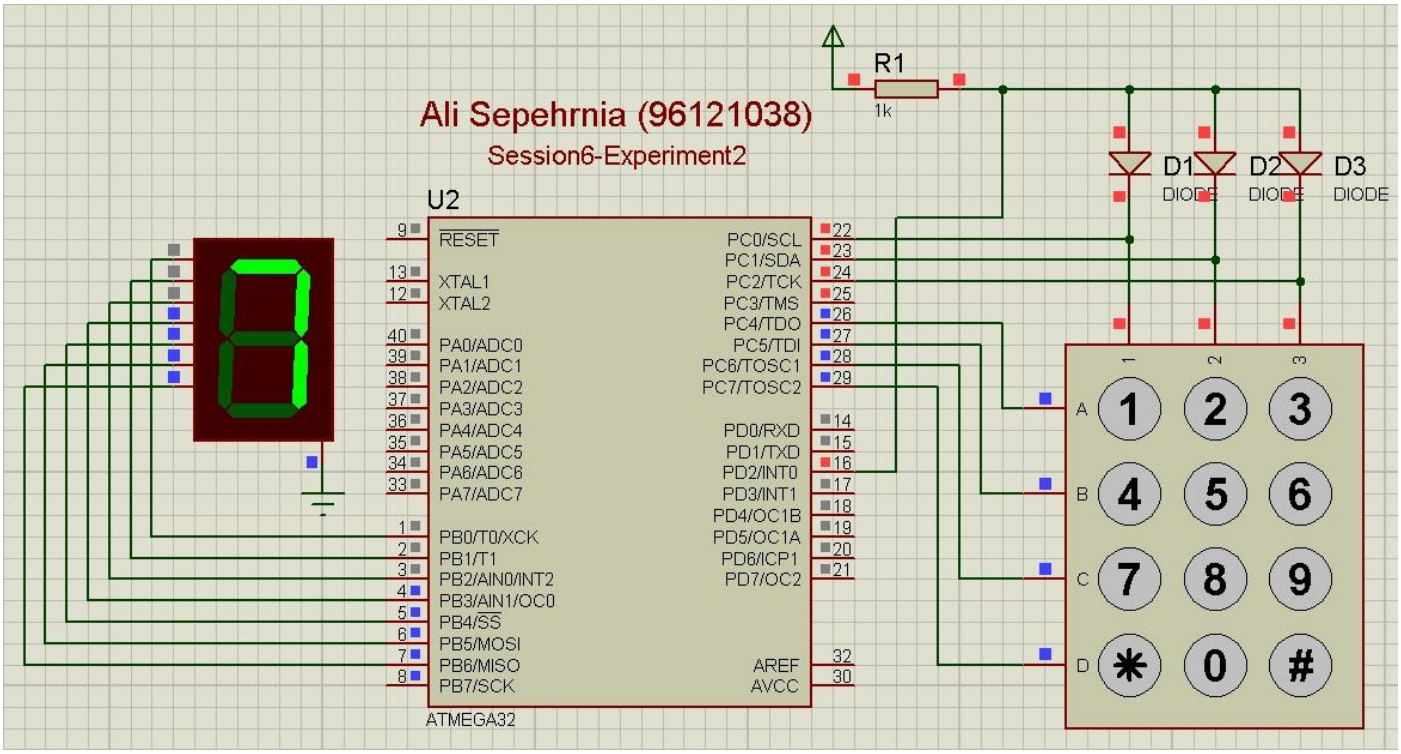
Notes | Az6-2.c

```
72              PORTB = sevenSegment[k];
73      }
74  GICR |= (1 << INTF0);
75  }
76
77  void main(void)
78  {
79  // Declare your local variables here
80
81  // Input/Output Ports initialization
82  // Port A initialization
83  // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
84  DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
85  // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
86  PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
87
88  // Port B initialization
89  // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
90  DDRB=(1<<DDB7) | (1<<DDB6) | (1<<DDB5) | (1<<DDB4) | (1<<DDB3) | (1<<DDB2) | (1<<DDB1) | (1<<DDB0);
91  // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
92  PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
93
94  // Port C initialization
95  // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=In Bit2=In Bit1=In Bit0=In
96  DDRC=(1<<DDC7) | (1<<DDC6) | (1<<DDC5) | (1<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
97  // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=T Bit2=P Bit1=P Bit0=P
98  PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (1<<PORTC3) | (1<<PORTC2) | (1<<PORTC1) | (1<<PORTC0);
99
100 // Port D initialization
101 // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
102 DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
103 // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
104 PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (1<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);
105
106 // External Interrupt(s) initialization
107 // INT0: On
```

G:\Personal\Uni\Micro Lab\Micro\Az6-2\Az6-2.c

Notes | Az6-2.c

```
106 // External Interrupt(s) initialization
107 // INT0: On
108 // INT0 Mode: Any Change
109 // INT1: Off
110 // INT2: Off
111 GICR|=(0<<INT1) | (1<<INT0) | (0<<INT2);
112 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (1<<ISC00);
113 MCUCSR=(0<<ISC2);
114 GIFR=(0<<INTF1) | (1<<INTF0) | (0<<INTF2);
115
116 // Global enable interrupts
117 #asm("sei")
118
119 while (1)
120       {
121       // Place your code here
122
123       }
124 }
125
```


Ali Sepehrnia (96121038)
Session6-Experiment2
R1
1k
D1
DIODE
D2
DIODE
D3
DIODE
U2
RESET
XTAL1
XTAL2
PA0/ADC0
PA1/ADC1
PA2/ADC2
PA3/ADC3
PA4/ADC4
PA5/ADC5
PA6/ADC6
PA7/ADC7
PB0/T0/XCK
PB1/T1
PB2/AIN0/INT2
PB3/AIN1/OC0
PB4/SS
PB5/MOSI
PB6/MISO
PB7/SCK
PC0/SCL
PC1/SDA
PC2/TCK
PC3/TMS
PC4/TDO
PC5/TDI
PC6/TOSC1
PC7/TOSC2
PD0/RXD
PD1/TXD
PD2/INT0
PD3/INT1
PD4/OC1B
PD5/OC1A
PD6/ICP1
PD7/OC2
AREF
AVCC
ATMEGA32
1 2 3
A 1 2 3
B 4 5 6
C 7 8 9
D * 0 #